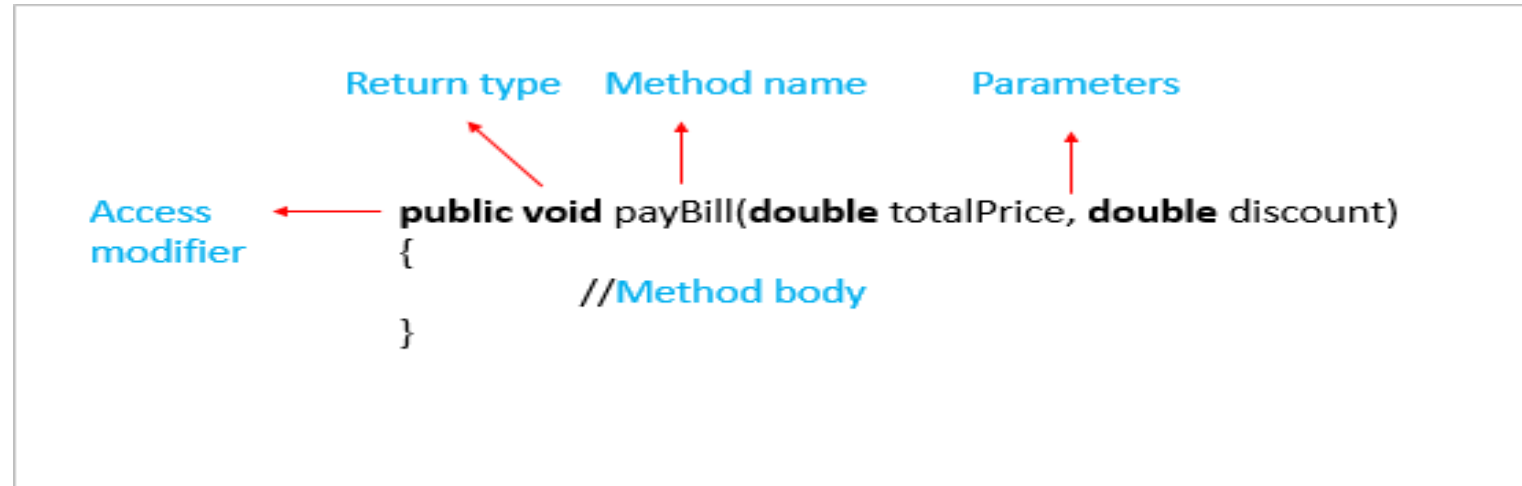


Method in java

A method is a set of statements which represents the behavior of a class. It is used to implement a specific functionality. In Java, every method must be part of some class.

Method syntax:



Access modifier – defines the access type of the method. You will learn more about this later in the course.

Return type – the data type of the value returned by the method or void if nothing is returned

Method name – name of the method

Parameters – comma separated input values passed to the method, should be () if no parameters are passed

Method body – the code that defines the functionality of the method

```
public class Customer {  
    public String customerId;  
    public String customerName;  
    public long contactNumber;  
    public String address;  
    public void displayCustomerDetails() {  
        System.out.println("Displaying customer details \n*****");  
        System.out.println("Customer Id : " + customerId);  
        System.out.println("Customer Name : " + customerName);  
        System.out.println("Contact Number : " + contactNumber);  
        System.out.println("Address : " + address);  
        System.out.println();  
    }  
}
```

```
public static void main(String args[]) {  
    Customer customer = new Customer();  
    customer.customerId = "C101";  
    customer.customerName = "Stephen Abram";  
    customer.contactNumber = 7856341287L;  
    customer.address = "D089, St. Louis Street, Springfield, 62729";  
    customer.displayCustomerDetails();  
}
```

```
public void payBill(double totalPrice, double discountPercentage) {  
    System.out.println("Calculating final amount to be paid.....");  
    double priceAfterDiscount = totalPrice * (1 - (discountPercentage / 100));  
    System.out.println("Hi " + customerName  
        + ", your final bill amount after discount is: "  
        + (int) (priceAfterDiscount * 100) / 100.0);  
}
```

```
public static void main(String args[]) {  
    Customer customer = new Customer();  
    customer.customerId = "C101";  
    customer.customerName = "Stephen Abram";  
    customer.contactNumber = 7856341287L;  
    customer.address = "D089, St. Louis Street, Springfield, 62729";  
    customer.displayCustomerDetails();  
    customer.payBill(500, 10);  
}
```

Displaying customer details

Customer Id : C101

Customer Name : Stephen Abram

Contact Number : 7856341287

Address : D089, St. Louis Street, Springfield, 62729

Calculating final amount to be paid.....

Hi Stephen Abram, your final bill amount after discount is: 450.0

```
public double payBill(double totalPrice, double discountPercentage) {  
    System.out.println("Calculating final amount to be paid.....");  
    double priceAfterDiscount = totalPrice * (1 - (discountPercentage / 100));  
    return priceAfterDiscount;  
}
```

```
class Demo {
    public int var1; // Instance variable of the class

    public void printValue() {
        int var2 = 20; // Local variable of the method
        System.out.println(var1);
        System.out.println(var2); // Local variable is accessible only inside
    // the method
    }
}

class Tester {
    public static void main(String args[]) {
        Demo demo = new Demo();
        demo.var1 = 10; // Instance variable can be accessed from outside the
                        // class with the help of object

        demo.printValue();

        // Local variables cannot be accessed outside a method
        // Below lines will lead to a compilation error saying that var2 is not
        // a field or variable
        // System.out.println(demo.var2);
        // System.out.println(var2);
    }
}
```

You will now understand the different parameter passing techniques.

Whenever a value of a primitive data type is passed, the values are copied from the actual parameters to the formal parameters. This kind of parameter passing is known as **pass by value**. In pass by value, both the actual and formal parameters point to different memory locations and the values are copied in both the memory locations.

E.g.:

```
public class Demo {
    public void changeValue(int value1, int value2) {
        value1 = value1 + value2;
        value2 = value1 - value2;
        System.out.println(value1 + " " + value2);
    }
}

public static void main(String args[]) {
    Demo demo = new Demo();
    int x = 20;
    int y = 10;
    System.out.println(x + " " + y);
    demo.changeValue(x, y);
    System.out.println("After calling changeValue function");
    System.out.println(x + " " + y);
}
```

```
20 10  
30 20  
After calling changeValue function  
20 10
```

You can see that the values are changed only inside the method. This is because any changes made inside the method will be reflected only in the memory locations of the formal arguments and not in the memory locations of the actual arguments.

When an object is passed as a parameter, the formal and the actual parameters both refer to the same object and hence the same memory location. Therefore, the changes made inside the method to the formal parameters are reflected in the actual parameters also. This kind of parameter passing is known as **pass by reference**.

E.g.:

Observe the below updateContact method in Customer class which updates the contact number of a customer.

```
        public void updateContact(Customer customer) {  
            customer.contactNumber = 9786758497L;  
        }  
  
    public static void main(String args[]) {  
        Customer customerOne = new Customer();  
        customerOne.contactNumber = 7656434567L;  
        System.out.println(customerOne.contactNumber);  
        customerOne.updateContact(customerOne);  
        System.out.println("After calling updateContact method");  
        System.out.println(customerOne.contactNumber);  
    }
```

```
7656434567  
After calling update method  
9786758497
```

```
public class Tester {  
    public boolean checkTemp(int temp) {  
        if (temp > 100) {  
            return true;  
        } else {  
            return false;  
        }  
    }  
  
    public static void main(String args[]) {  
        Tester tester = new Tester();  
        if (tester.checkTemp(110)) {  
            System.out.println("Data is valid");  
        } else {  
            System.out.println("Data is invalid");  
        }  
    }  
}
```

Data is invalid

Data is valid

true

false

What will be the output of the below method?

```
public class Tester {  
    public void demoMethod() {  
        int varOne = 1000;  
        float varTwo = 20.00f;  
        double varThree = 0.0;  
        varThree = (double) (varTwo + varOne * (7.0 / 100));  
        System.out.println("varThree:" + varThree);  
    }  
  
    public static void main(String args[]) {  
        Tester tester = new Tester();  
        tester.demoMethod();  
    }  
}
```

varThree:20.0
varThree:90.0
varThree:1020.0
varThree:0.0

Identify the number of instances and local variables present in the Student class given below.

```
public class Student {  
    public int rollNo;  
    public String studentName;  
  
    public void findGrade(int marks) {  
        char grade;  
        if (marks > 90)  
            grade = 'A';  
        else if (marks < 90 && marks > 80)  
            grade = 'B';  
        else if (marks < 80 && marks > 70)  
            grade = 'C';  
        else  
            grade = 'D';  
        System.out.println(grade);  
    }  
}
```

2 instance variables, 2 local variables

3 instance variables, 1 local variables

2 instance variables, 1 local variables

4 instance variables, 0 local variables

What will the output of the below code?

```
public class Tester {  
  
    public int subtract(int a, int b) {  
        return (a - b);  
    }  
  
    public static void main(String args[]) {  
        int a = 10;  
        int b = 20;  
        Tester tester = new Tester();  
        int res = tester.subtract(b, a);  
        System.out.println(res);  
    }  
  
}
```

0

-10

10

Compilation error: the name of the formal and actual parameters must be the same

What will be the output of the below code?

```
public class Tester {  
  
    public int add(int a, int b) {  
        return (a + b);  
    }  
  
    public static void main(String args[]) {  
        int a = 10;  
        int b = 20;  
        int sum = 0;  
        Tester tester = new Tester();  
        tester.add(a, b);  
        System.out.println(sum);  
    }  
  
}
```

10

0

20

30



Data is invalid

Data is valid

true

false

Option Data is valid is Correct

Explanation :

The method returns true as the value of temp is 110 and Data is valid is displayed.

Option varThree:90.0 is Correct

Explanation :

First, division is performed, then multiplication and then addition.

Option 2 instance variables, 2 local variables is Correct

Explanation :

Instance variables are declared inside a class and are used for storing values of objects.

Local variables are variables declared inside a method. Even parameters are local variables.

rollNo and studentName are instance variables and marks and grade are local variables.

0

-10

10

Compilation error: the name of the formal and actual parameters must be the same

Option 10 is Correct

Explanation :

The value of a is 20 and the value of b is 10 inside the subtract method. So, the value returned by the subtract method is 10 which gets displayed in the main method.

10

0

20

30

Option 0 is Correct

Explanation :

The value of sum is not getting updated and hence, the value of sum is 0.

Problem Statement

Implement a class Calculator with the method mentioned below.

Methods

- `findAverage(int number1, int number2, int number3): double`

Method Description

`findAverage()`

Calculate the average of three numbers

Return the average rounded off to two decimal digits

Test the functionalities using the provided Tester class.

Sample Input and Output

Sample Input	Expected Output
12, 8, 15	11.67
10, 20, 30	20.0